

Microserveis

Eloi Puertas i Prats

Universitat de Barcelona

Grau en Enginyeria Informàtica

20-05-2026

Pregunta:

Què passa quan una aplicació distribuïda deixa de ser un únic servei i passa a estar formada per desenes o centenars de serveis independents?

Evolució dels sistemes distribuïts

Idea clau

Volem passar de:

- | una aplicació distribuïda única
- a
- | un ecosistema de serveis distribuïts independents.

Recordatori: Arquitectura multicapa

En una aplicació WEB tradicional acostumem a tenir:

- Presentació
- Lògica de negoci
- Persistència

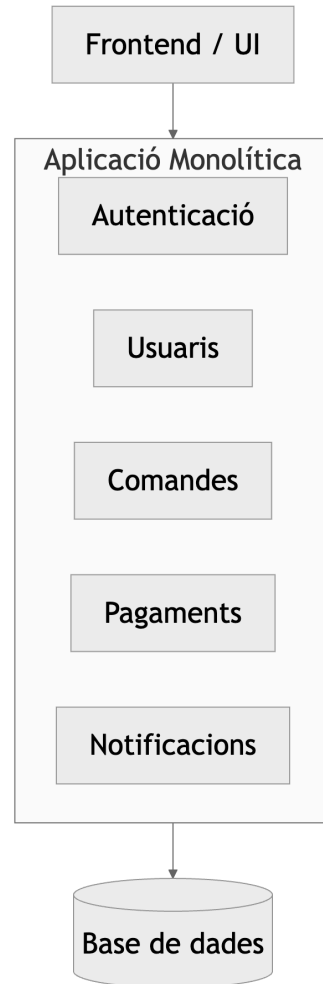
Arquitectura típica:

```
Frontend
|
Backend (negoci)
|
Base de dades
```

Problema:

| tot acostuma a desplegar-se conjuntament.

Aplicació monolítica



Tot el sistema és:

- un únic procés
- un únic desplegament
- una única aplicació

Aventatges del monòlit

Avantatges:

- Simplicitat arquitectònica
- Debugging senzill
- Un únic deploy
- Fàcil començar projectes petits
- Menys comunicació distribuïda

Ideal per:

- prototips
- sistemes petits
- equips reduïts

Problemes del monòlit

Escalabilitat limitada

Encara que només falli una part:

- s'ha de desplegar tota l'aplicació

Si una part consumeix més CPU que d'altres:

- s'ha d'escalar tot el sistema

Exemple:

- Pagaments saturat

↓

- Escalem tota l'aplicació

Problemes del monòlit (2)

Acoblament (coupling)

Tots els mòduls comparteixen:

- memòria
- procés
- cicle de vida
- dependències

Problema:

baixa independència entre components

Canvi petit:

Modificar Pagaments



Recompilar



Redeploy complet

Problemes del monòlit (3)

Tolerància a fallades limitada

Una excepció greu o consum excessiu de recursos pot afectar:

- Tot el sistema

Exemple:

- Bug a notificaciones



- Crash aplicació



- No funciona el sistema sencer

Solució: Microserveis

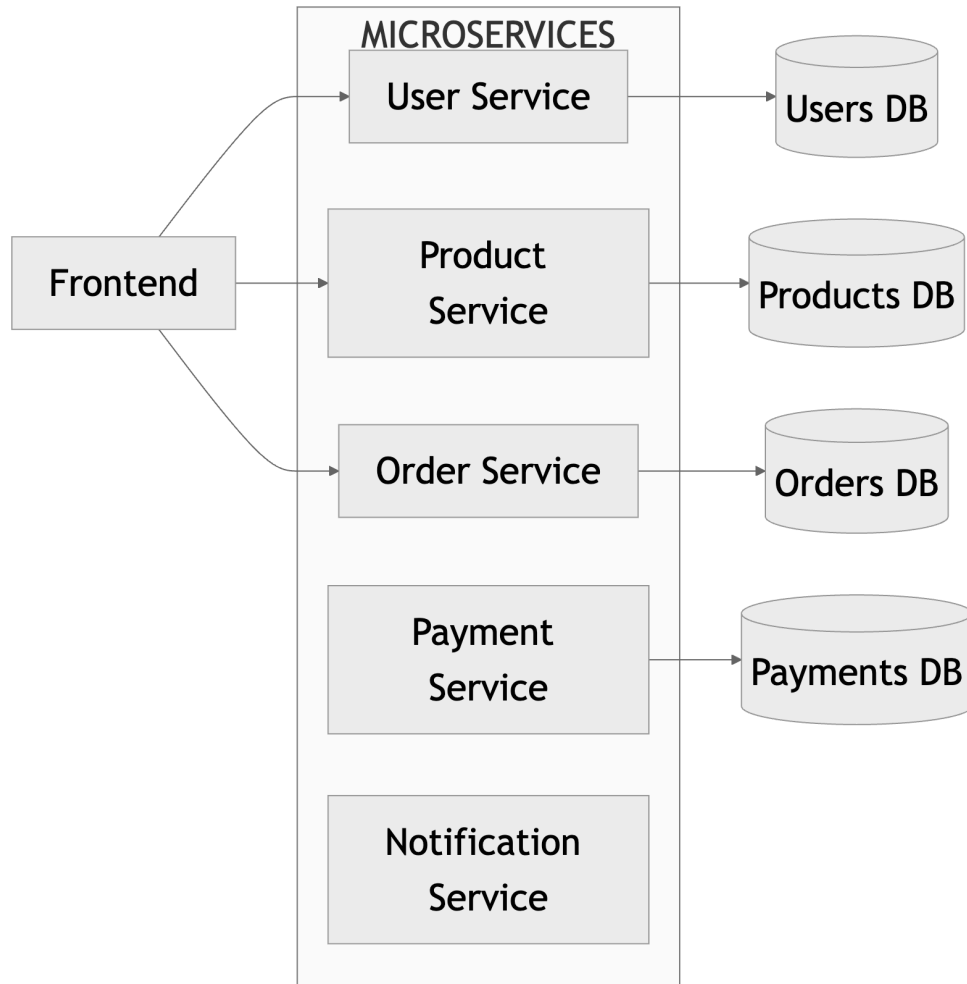
Idea bàsica

Dividir el sistema en serveis independents.

Cada servei:

- ha de ser un procés independent
- ha de tenir responsabilitats concreta
- exposa una API
- pot desplegar-se independentment

Arquitectura basada en microserveis



Idea important:

cada servei és independent.

Què és un microservei?

Un microservei és:

un servei petit, desplegable independentment, amb responsabilitat ben definida.

Característiques:

- procés independent
- comunicació per xarxa mitjançant APIs
- encapsula funcionalitat
- escalable individualment
- desacoblat dels altres serveis

Principis dels microserveis

Single responsibility

Cada servei fa una cosa ben definida

Exemple e-commerce:

- user-service
- order-service
- payment-service
- notification-service

Evitar:

Un únic servei de backend amb tota la lògica de negoci

Arquitectura distribuïda real

Cada microservei és:

- procés independent
- +
- API
- +
- persistència pròpia
- +
- desplegament independent

Important:

- No compartim directament memòria.

La comunicació és:

- per xarxa mitjançant APIs

Comunicació entre microserveis

1. Comunicació síncrona

Request / Response

Exemple:

- REST sobre HTTP

2. Comunicació asíncrona

Pas de missatges / esdeveniments

Exemple:

- Kafka

- RabbitMQ

3. Comunicació mesh de serveis

Descobrint de serveis i comunicació directa (P2P)

Exemple:

- gRPC

- Service Mesh (Istio, Linkerd)

Comunicació síncrona REST

Exemple:

Order Service necessita productes.

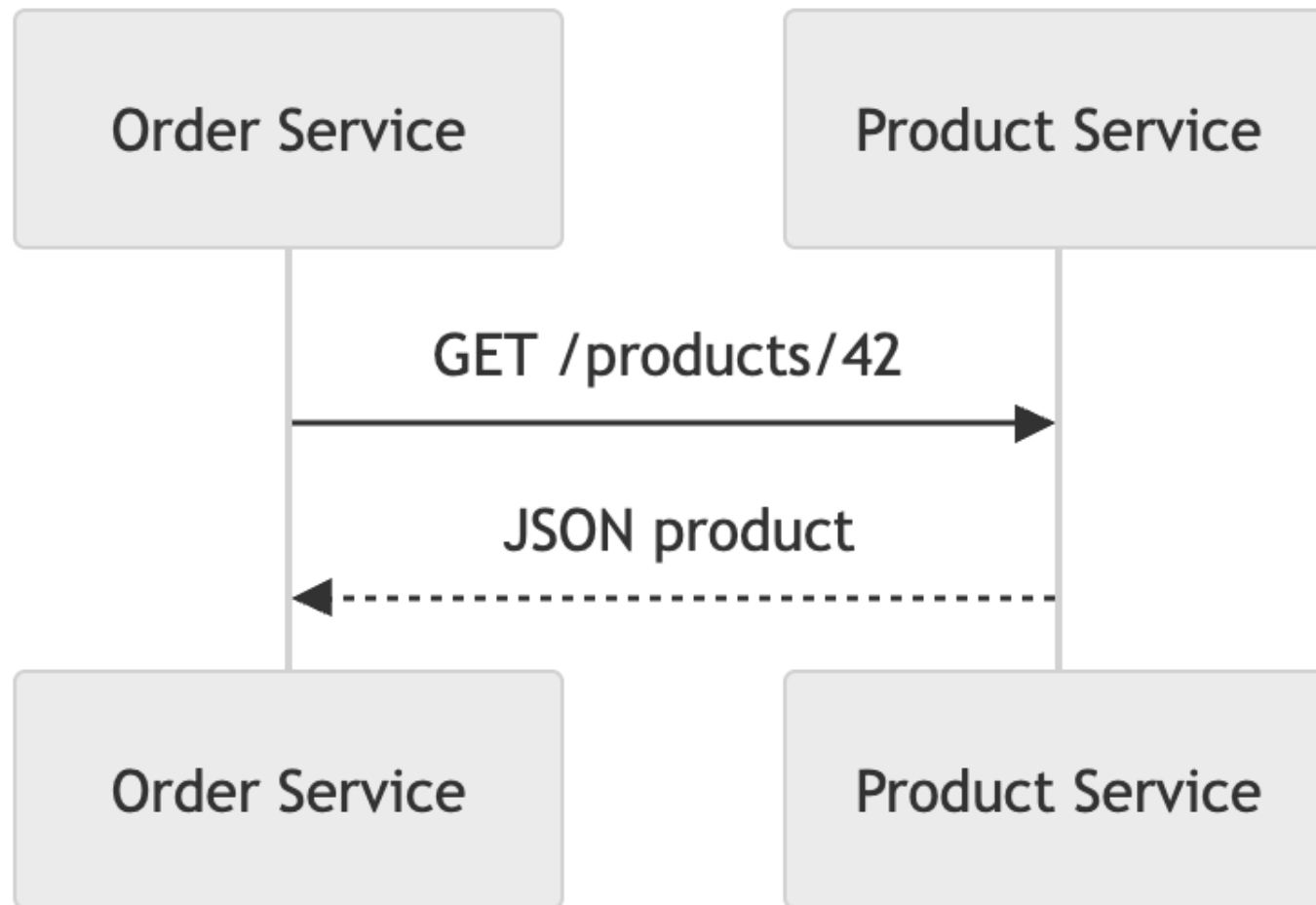
Request:

```
GET /products/42 HTTP/1.1
Host: product-service
Accept: application/json
```

Resposta:

```
{
  "id":42,
  "name":"Laptop",
  "price":899
}
```

Comunicació síncrona entre serveis



Aventatges del REST

Avantatges:

- simple
- conegut
- reutilitza HTTP
- interoperable
- fàcil de debugar

Ideal per:

- consultes
- operacions immediates
- APIs CRUD

Problema: Coupling temporal

Problema:

Order Service



Payment Service

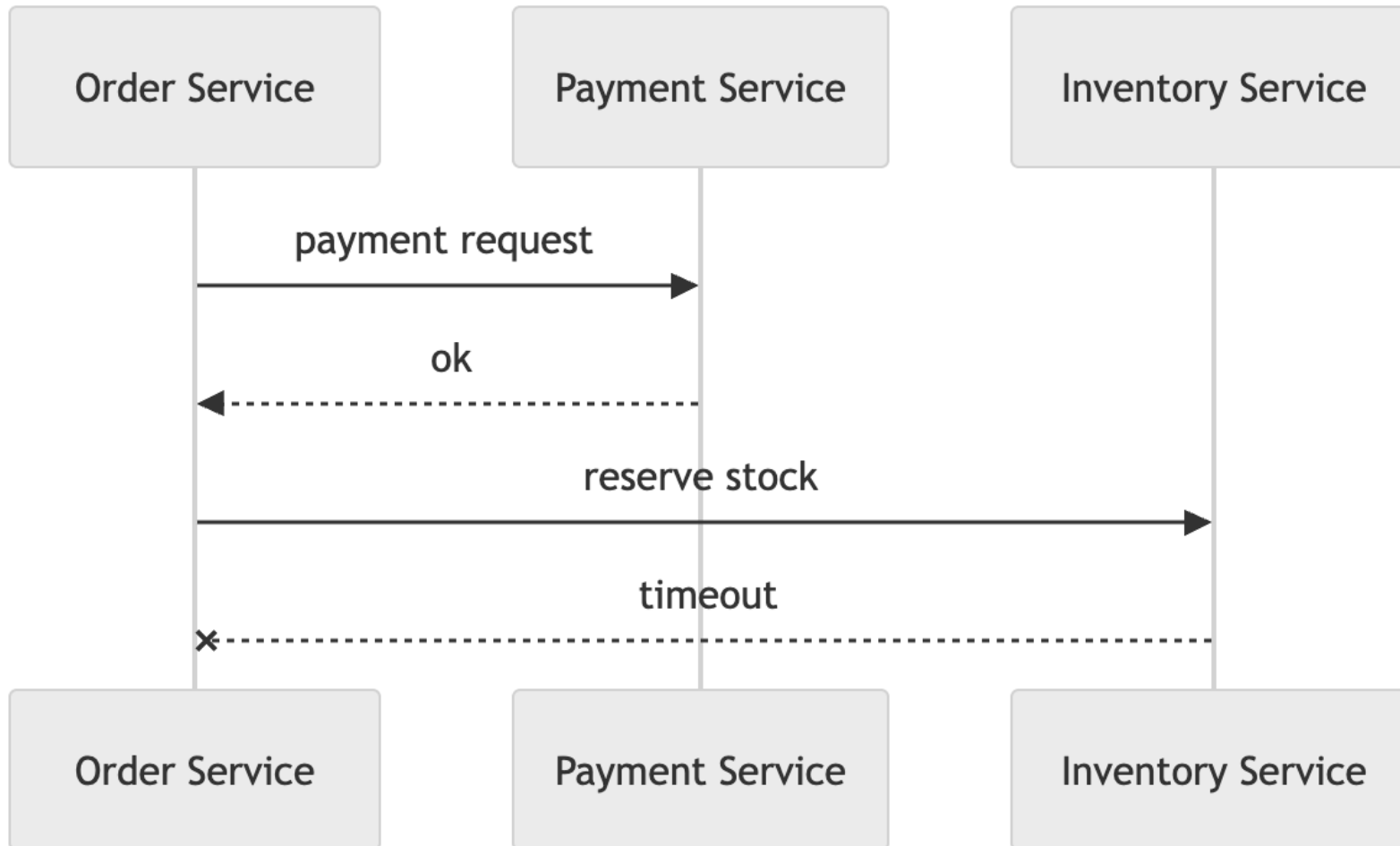


Inventory Service

Si un servei no respon:

cadena bloquejada

Exemple de fallada distribuïda



Problemes distribuïts reals

En microserveis apareixen:

- fallades parcials
- latències variables
- problemes de consistència
- errors de xarxa
- ...

El sistema:

- | no és una única màquina
- | sinó molts processos independents.

Solució: Tolerància a fallades

Tolerància a fallades:

- dissenyar el sistema per continuar funcionant malgrat fallades parcials.
- No es tracta d'evitar fallades, sinó de gestionar-les adequadament.

Estratègies:

- Timeouts
- Retries
- Idempotència
- Circuit Breakers
- Fallbacks

Tolerància a fallades: Timeouts

Com ja hem vist,

cap comunicació distribuïda ha d'esperar indefinidament

```
response = requests.get(url, timeout=3)
```


Tolerància a fallades: Retries

Si una comunicació falla:
...podem reintentar.

Request

↓ fail

Retry

↓ fail

Retry

↓ fail

Retry

↓ success

Però...

repetir operacions pot tenir efectes
secundaris

Tolerància a fallades: Idempotència

com ja hem vist, repetir operacions pot tenir efectes secundaris no desitjats.

Idempotència:

una operació és idempotent si es pot repetir diverses vegades sense canviar el resultat més enllà de la primera aplicació.

Exemple:

GET /products/42

POST /orders (amb un ID únic)

PUT /users/123 (actualitza usuari)

DELETE /sessions/abc (elimina sessió)

POST /payments (no idempotent, pot cobrar dues vegades)

Tolerància a fallades: Circuit Breakers

Si un servei falla repetidament:

- podem "trencar el circuit" i evitar més intents durant un temps.

- El sistema detecta fallades i deixa de fer peticions a un servei problemàtic durant un període de temps.

Exemple:

si el servei de pagaments falla, deixem de fer peticions durant 1 minut i mostrem un missatge d'error als usuaris.

Tolerància a fallades: Fallbacks

Si un servei no està disponible:

podem proporcionar una resposta alternativa o degradada.

Exemple:

si el servei de notificaciones falla, podem mostrar un missatge "Notificaciones no disponibles" en lloc de les notificaciones reals.

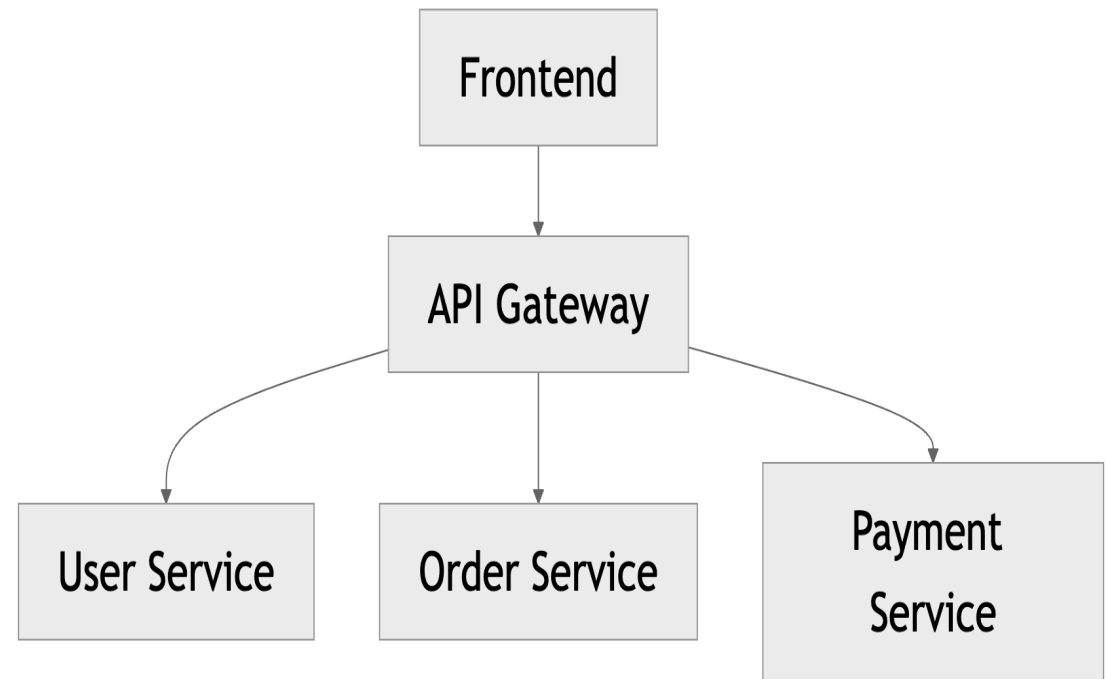
API Gateway

En una arquitectura de microserveis, el frontend no parla amb tots els serveis: és comú utilitzar un **API Gateway**

un punt d'entrada únic per a totes les peticions dels clients.

Responsabilitats habituals:

- routing
- autenticació
- rate limiting
- logs
- agregació de respostes



Arquitectura microserveis i pas de missatges distribuït

En lloc de comunicació síncrona, podem utilitzar un sistema de missatgeria distribuïda per a comunicació asíncrona entre serveis.

Avantatges:

- desacoblament temporal
- tolerància a fallades millorada
- processament en background
- escalabilitat

Exemple:

Order Service publica un esdeveniment "OrderCreated" en un sistema de missatgeria distribuïda

Payment Service consumeix l'esdeveniment i processa el pagament de manera asíncrona.

Event-driven architecture

Arquitectura basada en esdeveniments i missatges en lloc de crides síncrones.

- Un servei produeix **esdeveniments** quan succeeixen coses importants,
- i altres serveis **consumeixen** aquests esdeveniments.

Els serveis no es coneixen directament.

Què és un Esdeveniment?

Un esdeveniment:

és un fet rellevant ocorregut al sistema
descriu alguna cosa que JA ha passat

Exemples:

- UserRegistered
- OrderCreated
- PaymentCompleted
- ProductUpdated
- StockReserved

Exemples de tecnologies per a una event-driven architecture

- **Kafka**: plataforma de streaming de dades distribuïda
- **RabbitMQ**: sistema de missatgeria basat en cues
- **AWS SNS/SQS**: serveis de missatgeria gestionats al núvol
- ...

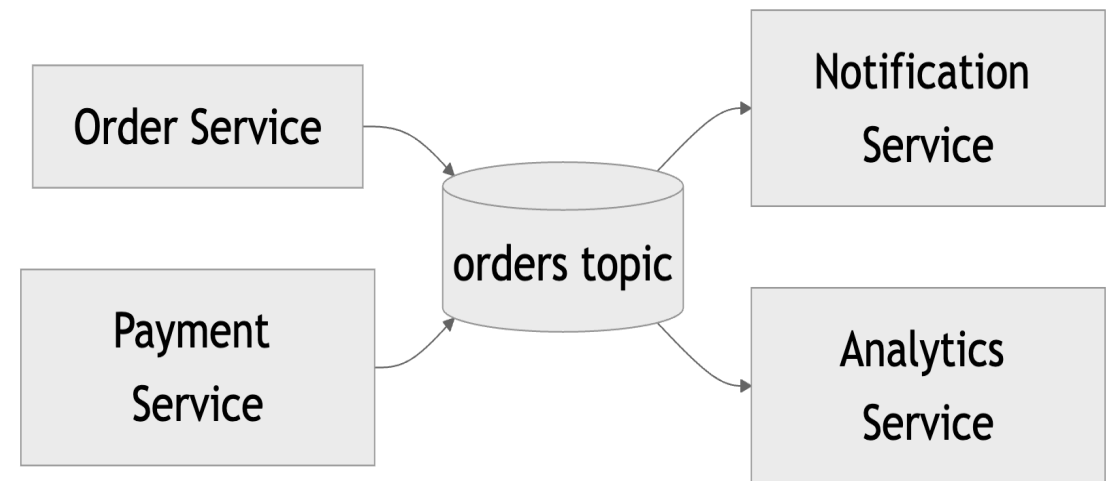
Comunicació basada en productor-consumidor

- Producer: publica esdeveniments a un canal o tema (topic)
- Consumer: s'inscriu a temes i processa esdeveniments quan arriben

Exemple:

Order Service (Producer) publica "Orders Topics" a Kafka

Notification Service (Consumer) s'inscriu a "Orders Topics" i envia notificacions quan arriben esdeveniments d'ordres.



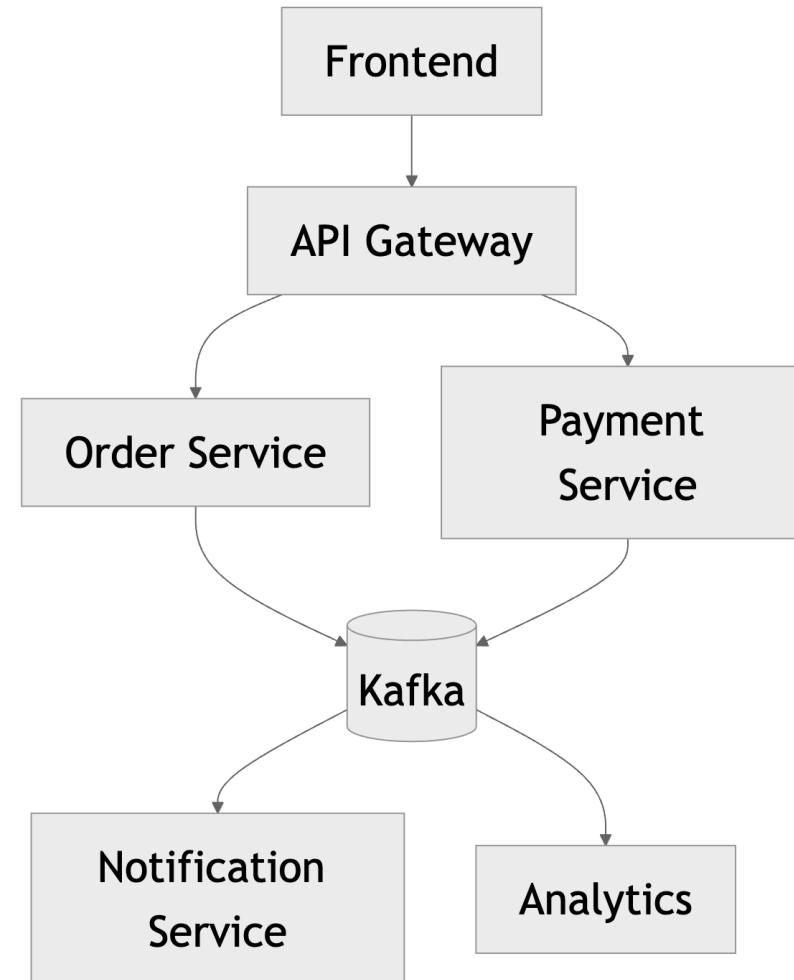
Arquitectura híbrida

El més habitual és combinar:

- comunicació síncrona (REST)
- comunicació asíncrona (esdeveniments)

per a operacions immediates

per a processos en background o desacoblats.



Mesh de serveis

- En lloc de tenir un API Gateway centralitzat, els serveis poden comunicar-se directament entre si mitjançant un mesh de serveis.
- El mesh de serveis proporciona descobriment de serveis, balanceig de càrrega, i comunicació segura entre serveis.
- Exemples: gRPC, Istio, Linkerd
- És similar a una arquitectura peer-to-peer entre serveis.

Desplegament de microserveis

- Cada microservei es desplega de manera independent.
- Normalment per gestionar el desplegament i l'escalat de múltiples serveis s'utilitzen:
 - contenidors (Docker)
 - i orquestradors (Kubernetes)
- Permet actualitzar o escalar serveis individuals sense afectar la resta del sistema.

Problemàtiques a tenir en compte en el desplegament de

- instal·lació
- dependències
- versions
- ports
- escalabilitat
- monitorització
- gestió de secrets
- gestió de configuració
- gestió de logs
- ...

Solució: Contenedors i orquestradors

- **Contenedors (Docker):** permet empaquetar un servei amb totes les seves dependències en una unitat desplegable.
- **Orquestradors (Kubernetes):** gestiona el desplegament, escalat i operació de múltiples contenedors en un clúster.

Contenidors: Docker

Un **container** és:

| una unitat executable lleugera amb aplicació + dependències

Conté:

- runtime
- llibreries
- configuració
- aplicació

Objectiu:

| mateix comportament a tot arreu

Virtual Machine vs Container

Containers:

- més lleugers
- arranquen ràpid
- menor consum
- menys aïllament
- menys segurs

Virtual Machines:

- més pesades
- arranquen lentament
- consumeixen més recursos
- més aïllament
- més segures

Arquitectura amb contenidors

- Cada microservei es desplega en un contenidor independent, que inclou l'aplicació i totes les seves dependències.

Exemple Dockerfile

```
FROM python:3.12

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["python", "main.py"]
```

Orquestració: Kubernetes

Necessitem:

| coordinar molts containers

Funcions necessàries:

- scheduling
- restart automàtic
- escalat
- networking
- load balancing
- observabilitat

Kubernetes: Què és?

Kubernetes és:

una plataforma distribuïda per orquestrar containers

Responsabilitats:

- desplegar serveis
- reiniciar processos
- balancejar càrrega
- escalar serveis
- gestionar xarxa

No executa:

una aplicació

Executa:

ecosistemes complets

Assumpcions errònies en microserveis

Assumir que:

- "és igual que un monòlit però separat"
- La xarxa és fiable
- latència zero
- ample de banda infinit
- cost zero de comunicació
- seguretat implícita

Però la realitat és molt diferent:

- fallades parcials
- latències variables
- problemes de consistència
- errors de xarxa
- ...

Quan no utilitzar microserveis?

- Sistemes petits o prototips
- Equips reduïts
- Baixa necessitat d'escalabilitat
- Baixa complexitat de domini

Quan sí utilitzar microserveis?

- Sistemes grans i complexos
- Necessitat d'escalabilitat individual
- Equips grans i especialitzats
- Necessitat de desplegament independent
- Domini amb responsabilitats ben definides
- Necessitat de tolerància a fallades millorada

Problemes dels microserveis

- Complexitat de la comunicació distribuïda
- Dificultat de debugging
- Gestió de dades distribuïda
- Sobrecàrrega operativa
- Dificultat de testing
- Dificultat de monitorització
- Dificultat de manteniment
- Dificultat en la traçabilitat del fluxe d'execució
- Es poden solucionar amb bones pràctiques i eines, però requereix més esforç que un monòlit.
- Algunes eines d'observabilitat i monitorització:
 - Prometheus
 - Grafana
 - Jaeger
 - Zipkin